

AI 研究者圆桌访谈中文翻译

参与者：Dwarkanish Patel；Beren Millidge（Zyphra CTO）；John Schulman（Thinking Machines 首席科学家，OpenAI 联合创始人之一，早期 RLHF 负责人）；Charlie O’Neill（Baseten 模型训练负责人）。

说明：保留原对话中的研究术语、观点归属、追问和分歧；中文按 AI 研究语境自然表达，必要时保留英文术语。

00:00:00 - 尽可能强化“RSI 不会发生”的论点

Dwarkanish Patel：今天我请来了三位我很熟悉的 AI 研究者朋友。每次和他们聊天我都能学到很多东西。而且他们所在的实验室和公司都算比较开放，所以很多事情可以公开讨论。今天的嘉宾有 Zyphra CTO Beren Millidge，Zyphra 正在开发开源模型；Thinking Machines 首席科学家 John Schulman，他之前是 OpenAI 的联合创始人，也是后来促成 ChatGPT 的 RLHF 工作负责人之一；以及 Baseten 的模型训练负责人 Charlie O’Neill。我第一个问题是：假设到了 2036 年，我们并没有看到数十亿个疯狂的超级智能到处运行，也没有发生那种彻底改变世界的事情。先排除政治冲击、战争、AI 被禁止之类的外部因素。最可能的技术原因会是什么？为什么 2036 年没有变成一个充满超级智能的世界？

Beren Millidge：这里一直有一种很经典的现象，有点像 Moravec 悖论。我们经常会想：“只要 AI 能做到这个，那一定就非常厉害了。”比如能解非常难的数学题，能赢国际象棋之类的。结果等 AI 真把这些事情解决以后，你会发现它并没有产生想象中那么大的影响。当然会有影响，但没有那么彻底。如果这种情况一直持续下去，始终没有出现真正强的 generalization，我觉得可能会出现这样一种情况：AI 对 benchmark 里的东西、训练 environment 里的东西都极其擅长，但始终存在一个顽固的 sim-to-real gap，把真正的应用挡在那里。我觉得这种情况概率不算高。因为现实里，即使只是 RL，我们现在也确实已经看到了一些 generalization。但如果 meta-learning 真的特别难泛化，再加上 continual learning 一直解决不了，而且本身就是个极难甚至近乎不可能的问题，那么这可能就是我默认会想到的那个失败场景。

John Schulman：我同意。现在人类相对模型还是有很多优势。每次新模型出来以后，它都会在其中一些方面追上来。但真正做事情的时候，最后往往还是会被模型比较弱的地方卡住，比如 judgment 比较差，或者模型没办法足够可靠地检查自己的工作。这个循环已经重复很多次了。一个新模型出来，大家都很震撼，说：“这次就是 AGI 了。”然后用一个月左右，又开始觉得它很笨。这个循环可能还会重复很多次。到底还要重复几轮其实很难预测。现在之所以还没有出现 capability 的爆炸式增长，就是因为在做研究和工程的时候，模型仍然有足够多的 bottleneck。即使模型写代码的量能比人多很多，也不意味着你的生产率就能提升 100 倍。所以也许只是这种循环还要再持续很多轮，比我们原来预想的次数更多。

Charlie O’Neill：对我来说，核心问题是：如果考虑“理论上能装在芯片上的最优 learner”，那么现在的 Transformer + RL 这套 recipe 离那个 global optimum 到底还有多远？大家通常会想象，一旦我们有了一个 AI research agent，它比所有人类 AI 研究者都强，哪怕只强 0.1%，那么你可以同时运行几十万个、甚至几百万个这样的 agent。而且随着芯片越来越快，这些 agent 的运行速度还会继续提高。这样一来，并行规模最终可能压过其他所有 bottleneck，然后你就会进入非常快的 self-improvement，也就是 fast takeoff。但我能想象另一种情况。如果我们继续沿着现在这条路线走，本质上就是 self-attention、RL，再不断扩大 RL environment。可以类比一下 Moore's law。我们当时看到的是一条非常漂亮的直线，而且持续了非常长时间。但要让那条 scaling law 一直维持下去，中间其实发生了很多离散的技术突破和创新。LLM 也是类似。一开始有 pre-training scaling law，然后开始出现 diminishing returns。接着我们引入 RL，又把这条曲线接了下去。于是从外面看，它仍然像一条持续上升的直线，但中间其实已经切换过增长机制。所以如果接下来还需要再出现一次这种 discontinuity，我并不确定现在这种“用 RL environment 训练 LLM”的方法，包括专门针对 RSI 的 RL environment，是否真的有能力自己发现那个 discontinuity。如果发现不了，我们可能最后就会进入某种 asymptotic curve。

Dwarkanish Patel：但你觉得下一次需要的 discontinuity，会比 2012 年以来发生过的任何突破都更难吗？

Charlie O’Neill：如果我们知道答案，可能也就已经知道该怎么实现它了。不过可以区分两类 discontinuity。一种是在现有 paradigm 上继续叠加，是 cumulative 的。比如 RL 之后还有一个我们没发现的东西，但它仍然能接在现在这条线上。这种东西也许现有 AI 能够把线索连起来并发现。另一种问题是：我们现在离 global optimum 到底有多远？会不会必须退回去，连 gradient descent 和 neural network 本身都扔掉？如果真正需要的是这种距离非常远的新范式，我不觉得只要继续 scale 当前 paradigm，哪怕运行再多 LLM，就一定有能力发现它。

Dwarkanish Patel：所以唯一真正的希望，大概就是 deep learning 本身根本无法把我们带到这样一个 AI：它至少在 research 和 development 上全面压过人类，包括人类提出全新 paradigm 的能力。当然，也有可能即使是人类自己，永远也发现不了下一个

learning architecture。但如果假设人类最终可以发现，那只要看看从 2012 年到现在的进展，再把这条趋势往前推……我知道这里面很大一部分来自 compute scaling，但如果未来几年它就是始终到不了能够在 R&D 上压过人的程度，感觉还是挺奇怪的。Ryan Greenblatt 最近也上过这个节目。他提过一个观点，我也很想听听你们怎么看。随着 AI 越来越强，可以想象它能够在 simulation 中持续取得进展，而这些 simulation 不只是训练 AI R&D，也训练更广泛的科学研究能力。现在所有大实验室，以及很多创业公司，其实都在做类似的事情。还有一个直觉例子是 chess bot。如果看从 1980 年代开始 chess engine 的 Elo，你会发现它其实长期是比较线性地上升。但当它穿过 human range 时，却出现了一个非常巨大的实际变化：之前是人类专家总能赢 AI，之后变成了人类专家几乎永远赢不了 AI。虽然底下只是 Elo 的线性增长，但跨过人类水平以后，系统表现出来的结果是高度不连续的。我同意你前面说的：到目前为止，AI capability 对现实经济造成的最终影响还没那么夸张。但可能只是因为它相对人类的 Elo 还在慢慢往上爬。

Beren Millidge: 对，我也觉得如果它真的停下来，会非常令人意外。要让你描述的那个场景成立，也就是到了 2035 年一切还基本正常，我觉得唯一的技术可能就是能力曲线恰好在人类水平之前 asymptote 掉了。因为按我的判断，我们现在已经很接近开始跨过 human Elo 的位置了。所以它必须刚好在那之前开始变平。除此之外，另一个更可能的原因反而是 AI 受到非常强的监管限制。实际上，如果真要我选，我觉得监管比技术原因更可能导致这种结果。

Charlie O' Neill: 我觉得 research 本身其实有不同类型。有一类类似 autoresearch: objective 已经定义得非常清楚，你只是不断优化这个 objective。大家现在想象的未来，大概就是继续让 pre-training loss 往下降，让 environment 里的 reward 往上升，于是能力持续提高。但 Ryan 说的可能是另一类更加 open-ended 的科学问题。真正产生 paradigm shift 的时候，我们往往根本无法提前定义 objective。而且 AI 自己显然现在也没有能力可靠地提出那个 objective。所以对于这些事情，我们必须非常谨慎地考虑 objective 到底怎么定义。

Dwarkesh Patel: 也许你的意思是，从 2012 年以来真正发生的那些突破，其本质就是我们不断发现了新的 objective 或 inner loop。比如在 2014 年，大概不会有人觉得，未来最重要的事情之一是做 nanoGPT speedrun。但等 next-token prediction 这个 paradigm 被发现以后，你当然就会想到：那让 AI 把这个 inner loop 做得极其高效就好了。可是也许下一步还有另一个我们现在根本想不到的 inner loop，所以 AI 自己也不知道应该优化什么。虽然外面可能还有一个很慢的 outer loop，比如 revenue，但那个反馈太慢了。John，你当年就在第一线。你怎么看？

John Schulman: 其实我记得在 OpenAI 很早的时候，我自己就有一种直觉：仅仅 minimize log loss 应该不可能得到真正的 intelligence。因为真正重要的那些信息，只占整个 loss 的极小一部分，很容易被大量 noise 淹没掉。所以我当时会觉得，只训练 language model 做 next-token prediction，应该学不到我们真正关心的那些东西。我们需要设计更好的 objective，让模型更加重视真正重要的部分。你可以为这个观点构造各种各样的理由。比如，人类显然也不会精确地建模环境中的一切。大多数人看过一个场景以后，也画不出 photorealistic 的复原图。所以我们可能需要一个比“预测所有东西”更合理的 objective。结果最后事实证明，next-token prediction 居然就是有效。

Dwarkesh Patel: 你刚才也提到了，即使在今天的 AI research 里，post-training benchmark 这种 inner loop，最后也未必和用户真正喜欢的东西一致。

John Schulman: 是的。整个领域其实极其依赖 generalization，而我们很难预测什么时候会发生 generalization，什么时候会出现 out-of-distribution generalization。我们知道，如果直接在你关心的任务上训练，模型肯定会变得更好。但很多最重要的进步，恰恰来自一些我们事先根本没有理由相信会发生的泛化。比如，仅仅在非常朴素的 next-token-prediction objective 上做 pre-training，却泛化到了各种需要深度理解输入的任务。或者模型从 pre-training 里学会了一种能力，而这种能力在训练数据里其实非常少见。还有从 verifiable task 泛化到 less-verifiable task，这同样是一种事先没有理由保证会发生的 generalization。

Dwarkesh Patel: 这就引出一个很有意思的问题。为什么有人会觉得 singularity 可能很快发生？甚至都不需要增加 AI progress 里除了 AI labor 之外的其他投入。可以想象，以后每次你准备跑一个耗资百万美元级别的 experiment 之前，先投入同等规模的 compute，让 AI researcher 做思考。相当于有自动化版本的你们，在真正跑实验之前先思考一百年：这个实验到底应该怎么设计？做各种 small-scale ablation，甚至重新构建从 deep learning 之前开始的理论体系。等真正准备跑实验的时候，setup 已经接近最优。然后实验跑完以后，再让这些 AI 花一百年分析结果，决定下一个实验应该是什么。

John Schulman: 如果你真的能够思考足够久，也许很多事情其实事先是可以推出来的。可能存在某种非常聪明的 small-scale experiment，让你先在小规模上建立理论，然后理论能够 generalize 到大规模实验。所以我觉得我们距离“研究本身可以被做到多好”的 ceiling 还非常远。我能想象未来 AI 会把大量 compute 用在分析和 theory building 上。也许用在思考、分析实验、构建理论上的 compute，会和真正用来跑 experiment 的 compute 一个量级。

Charlie O' Neill: 我觉得如果 objective 定义得很清楚，这里面已经有很多非常具体的例子。单纯思考能做的事情，本质上是根据你形成 prior 以后新获得的 bits 去 update posterior。仅仅靠思考本身，你并不会凭空获得新的 bits。但如果 objective 已经定义好了，同时手上又有大量数据，那么我确实能想象当前 paradigm 会出现很大的加速。一个很好的例子就是 Kaplan scaling laws。如果当时有今天这种 AI 去认真检查那些结果，它可能很早就发现：他们直接用了 intermediate checkpoint，却没有正确考虑 annealing，所以这里其实有问题。这种事情可能几年以前就被发现。光这一点，也许就能把一两年的研究进度直接省掉。类似的还有 μP 、learning rate 应该怎样随 model size scaling，以及 model width 在这里面的影响。如果 objective 已经明确，比如“把 pre-training loss 做得更低”，我觉得 AI 能重新推导出大量东西，消除很多 low-hanging fruit。如果任务只是“最大化我们现在已经知道的 objective”，我完全可以想象 10 倍级别的加速。但我完全看不出来，这怎么能自然泛化到“首先找到正确的 objective”。思考得更多，并不会自动让你得到正确的 objective。

Beren Millidge: 我觉得这实际上就是当前 AI 要实现快速 RSI 时最关键的问题：AI 到底能多好地泛化到学习自己的 objective？如果要形成一个真正 self-propelling 的自动化循环，AI 必须能够：提出一个 objective，优化它，理解发生了什么，再提出下一个 objective。而且这个循环要持续很长时间，中间任何一步都不能跑偏。回到 Moravec 悖论。也许这里同样存在一个 Moravec-style 的问题。我们人类每天都会自己想“我下一步该做什么”，然后去做，再根据结果继续决定，所以我们觉得这种自主闭环非常自然、很简单。毕竟 evolution 也必须产生能长时间自己生存的生物。但也许对 AI 来说，这件事情就是莫名其妙地特别难。就像 locomotion 对机器很难，而数学反而非常容易，虽然对人类恰好相反。

Dwarkesh Patel: 但现在 agent 的 time horizon 一直在变长，这不是说明——

Beren Millidge: 对，所以我也同意，目前并没有明显证据支持刚才那个悲观猜测。实际上，现在的 agent 已经可以非常 persistent，而且让它持续做事情已经变得比较容易了，这反而算是反对那个观点的证据。但如果这种完全自主的长期闭环真的非常难，它仍然可能成为“不发生快速 takeoff”的原因之一。

Dwarkesh Patel: 如果从 2012 年一直看到现在，或者从你们开始研究 AI 一直到现在：所有这些 innovation 里，包括纯 engineering 的，也包括纯 conceptual 的，哪一项最可能成为“在人类彻底退出 AI R&D 自动化之前，最后还需要由人类完成的事情”？

Beren Millidge: 可能就是不断提出正确的问题。即使 AI 能替你做任何 experiment，你还是得决定应该做什么 experiment。至少现在，AI 在这方面远远不如它写实验代码那么好。每次你和模型聊 research，它经常会提出一大堆杂七杂八的方向，但基本都是很小的 incremental step。

Charlie O' Neill: 甚至可以看 DeepMind 当年的路线：“我们要通过学会以超人水平玩游戏来解决 intelligence。”然后另一边，有 Radford 这样的研究者突然说：“我要不就拿特别广的数据，直接预测 next token 试试看。”即使 Radford 已经发现这个方向以后，也过了很长时间大家才真正决定把它 scale up。因为后来还得出现 scaling laws 这个想法，也就是意识到这种东西的增长可以如此稳定地预测。

John Schulman: 我觉得最后还会保留给人类的工作，或者说存在时间最长的人类角色，应该是：定义 objective，以及决定我们到底想要什么。比如，AI assistant 应该表现成什么样，什么叫 helpful，RLHF 的 objective 到底应该是什么。后来又出现 constitution、model spec，这些本质上都是同一种问题。即使 AI 可以做所有技术工作，人还是必须在很大程度上决定：我们究竟希望系统做什么。

Dwarkesh Patel: 所以 alignment 是最后一份工作。

John Schulman: 差不多。不过 alignment 本身也可以拆成两部分：第一，specify objective，或者说确定正确的 objective 到底是什么。第二，真正实现并优化你已经定义好的 objective。我觉得第一部分不会很快消失。如果看一个 post-training team 为什么需要这么多人，很大程度上就是因为有太多不同领域，都需要有人决定模型在这里应该怎么表现。整件事情非常难彻底自动化，因为总得有人思考：在这个场景里，我们究竟希望模型怎么做。

00:18:39 - 中国实验室的进步主要由什么推动？

Dwarkesh Patel: 为什么现在 model provider 没有出现非常严重的集中化？这里明明有很多力量都应该把市场推向 centralization。如果从几年尺度来看，有什么东西能阻止这种趋势？

John Schulman: 我觉得最主要的是 distillation。基本上，任何能通过 RL 学到的东西，通常都很容易被 distill。因为这些行为本身需要的信息量并不是很大。如果你能够从模型那里拿到展示这种行为的 trajectory，那么通常就比较容易把这种行为蒸馏出来。所以我觉得 distillation 是对抗 centralization 的一个重要力量。还有一种可能是出现 company-specific model。公司可以

不断从自己的 deployment 中学习，让自己的模型持续提高。提供这种系统的，可能仍然是今天这些寡头 model provider，也可能是现在规模较小的其他公司。但无论如何，我觉得这都会改变整个格局。

Beren Millidge: 还有一点，continual learning 本身其实也阻止不了 distillation。就算你的模型每天都在进步，别人也可以每天都重新蒸馏。这两个 loop 完全可以以同样的速度运行。

Dwarkesh Patel: 有道理。所以如果想复制模型行为，你首先得知道应该从什么 prompt distribution 里采样，才能把真正重要的行为 elicited 出来，对吧？

John Schulman: 对。如果只是用 supervised learning 做 distillation，prompt distribution 极其重要。即使你能完全访问 teacher，甚至连 chain of thought 都能拿到，也不代表把模型完整蒸馏出来是一件简单的事。要把所有有用能力都蒸馏出来，非常不容易。因为你必须给模型 prompt。而且必须是真实的 prompt。你需要一个覆盖面非常广、又足够接近真实用户分布的 prompt distribution。最近有一种说法是，一些中国公司可能在使用某些 router service。这些服务本来的用途，是让中国用户通过代理访问本来在中国无法直接使用的美国 frontier model，主要还是 coding 使用。这些 router 或 proxy service 会收集部分数据，甚至出售其中一些数据。这种数据对 distillation 非常有价值，因为它几乎直接给了你真实的 prompt distribution。

Beren Millidge: 我觉得这里 AI 本身能帮很大的忙。如果去看 frontier pipeline，或者看中国模型公开在论文里的做法，通常都是先从某些地方得到 seed prompt。这些 seed 可能来自人类，也可能来自刚才说的真实数据。然后再用现有模型，或者其他 frontier model，从这些 seed 出发合成覆盖范围极大的 prompt。所以 prompt distribution 的扩展、environment creation，其中大量步骤都可以自动化。随着模型越来越强，人类需要提供的 bits 会越来越少。

Dwarkesh Patel: 但你最终不还是被一个问题卡住吗：你必须真的有一个用户服务，有大量用户通过它产生数据。

Beren Millidge: 不一定。当然有真实用户非常有帮助。但理论上，你也可以直接去想象用户会想要什么。

Dwarkesh Patel: 可问题就在于，真实用户不是只说：“帮我做一个这样的 app。”他们可能接着说：“不对，这个没用，我其实想加另一个 feature。”然后又说：“等等，还是先退一步，换一个方向。”真正重要的是捕捉这整段 trace。如果 AI 自己本来就能模拟这些复杂 trace，那其实你基本已经有 RSI 了。

Beren Millidge: 对。如果真的有一个完全自动化的 loop：AI 自己决定数据是什么，也自己决定怎么训练。那本质上就是 RSI。所以真正的问题是：你到底还需要多少来自人类的信息。到某个阶段，也许你只需要告诉模型：“我想要一批大概长这样的 trace。”然后模型自己就能生成相当不错的近似。

Dwarkesh Patel: 但比如我说：“我要训练一个很好的政治人物。”那模型必须凭空预测 Senate 走廊里的真实讨论会是什么样。我感觉还是有很多这样的东西——

Beren Millidge: 有意思的是，这种事情对 distiller 反而可能比对 frontier lab 更容易。Distiller 只需要说：“我要一个优秀政治人物的行为。”然后去问 frontier model。如果 frontier model 已经知道优秀政治人物应该怎么做，它直接生成这些 trace 就行。但如果你是在第一次训练出这种能力的 frontier lab，你得真的获得政治人物每天在干什么的数据。所以从“已有一个会做的人，然后复制它”出发，实际上比第一次把这个能力创造出来容易得多。你只要告诉模型：“给我这种东西。”然后生成十亿个变体，比第一次从零把这个能力造出来容易很多。

Charlie O' Neill: 我觉得，如果我们接受中国实验室确实拿到了大量 router data 这个观察，其实可以推出一个挺具体的结论。我之前就在想：为什么 Sonnet 5、Opus 5，看起来几乎客观上比 GLM-5.3、Kimi K3 更差？这挺奇怪的，因为 Anthropic 不只有 distillation，甚至还能从更强模型做 logit distillation。一种解释就是：prompt distribution 真的非常重要。你必须知道真实用户到底在干什么，才能把这些行为真正 distill 进来。从这里甚至可以推出一个更大胆的判断：frontier lab 在 RL environment 上未必还有想象中那么大的优势。当然，真实用户分布对 general behavior 很重要。但按理说，如果要测某种 capability，frontier lab 自己造出来的那些最难 RL environment 应该已经很好了。如果 Anthropic 有自己的最前沿 RL environment，又可以做 logit distillation，结果做出来的模型仍然更差，那也许——

Dwarkesh Patel: 也许 real-world deployment 比 environment 本身更重要。这个挺有意思。但最初这些 capability 毕竟还是在 Fable 或更强 frontier model 里通过训练被激发出来的。所以很奇怪，为什么你不能在更小模型里重新通过 incentive 把它们激发出来。

Charlie O' Neill: 也许我们现在正好处在一种很奇怪的 uncanny valley。你过度去复制一个 frontier model，但是 student-teacher gap 又太大。很多人对 Opus 也有类似看法。Opus 4.6 到 Opus 5 的变化，有一种感觉是：Opus 5 像是一直有一个 AI-

as-a-judge, 在检查自己做过的每个决定。所以它会花很多 token 去考虑所有可能的问题。但它未必拥有像更大模型那种“什么时候该停”“哪条路线值得继续”的直觉。

Dwarkesh Patel: 有点像能力追求超过了自身真正掌握的程度。

John Schulman: 我会提供一个稍微不同的解释。我觉得 environment 至少可以沿两条轴来考虑。一条是 difficulty, 另一条是 realism。Difficulty 比较容易做高。你可以设计特别复杂、特别需要技巧的任务。我有时会把这叫做 benchmaxxing distribution: 现在很多著名 benchmark, 其实就是非常难的 puzzle-like task, 但同时又非常容易 verify。另一条轴是 realism。比如真实 coding agent 的工作环境, 会有人类来回沟通, 目标也不止一个, 任务会发生变化。真正第一次创造模型行为的 lab, 必须同时沿着这两条轴推进。如果想让模型在真实场景里表现好, 就必须在 realism 这条轴上投入很多工作。可能需要 rubric, 或者 human feedback, 去构造对应 reward function。但如果你只是 naive distillation, 最后往往只是在 benchmaxxing distribution 上把 teacher 学得很好。如果你的训练 environment 没有覆盖那些真正复杂、现实的能力, 那么这些东西就不会进入 student model。也许现在发生的一件事情是: 大模型能够从那些狭窄但很难的任务泛化到更真实的任务。如果你能拿到非常好的 realistic prompt distribution 去做 distillation, 就能把大模型复制得相当好。但如果你只有大量容易验证的任务, 那么学生模型可能 benchmark 全都很高, 在更广泛的真实分布上却明显更差。这甚至可能解释 Anthropic 的一些小模型, 比如 Sonnet 5。当然, 我们不知道他们具体怎么做 post-training。也可能只是他们一直在修改自己的 post-training stack, 而某一代调坏了某个东西。也许某个参数推得太高, 导致模型出现一些用户很不喜欢的行为。Post-training 非常容易在 benchmark 看不出来的地方把模型搞坏。

Beren Millidge: 还有一个非常基础的点。Frontier AI lab 会从大型数据公司买数据。中国公司同样也可以买到这些数据, 而且他们确实在买。所以如果双方最终拿到的是同一批数据, 再加上 distillation, 那想要一直跟上其实并没有那么难。

00:28:06 - 能够自动化 AI research 的模型会怎样训练出来?

Dwarkesh Patel: 我还有一个问题: 第一批真正能够自动化 AI R&D 的模型, 到底会怎样被训练出来? 有一个 toy version。比如 Ryan 提过, 可以让 GPT-8 去训练 GPT-3 规模的模型, 然后给它一些 inner-loop challenge: 比如学习需要 continual learning 的电子游戏; 或者要求它用尽可能少的 compute, 把 loss 降到某个值。John, 你之前提过一个很有意思的观点: 真实情况可能不会这么训练出来的。所以如果未来真的有 AI 可以自动化 AI R&D, 它大概率经历了怎样的训练过程?

John Schulman: 我觉得大概会结合两种东西。一种是从 human feedback 中学习, 把研究者的 judgment 或 taste 吸收进去。另一种是创造大量 practice environment, 让模型做完整的 multi-step research project。现实里大概率就是这两种一起用。然后每一代模型出来以后, 看上一代哪里最坏, 就专门 patch 哪里。研究人员会大量使用 AI, 然后发现它有一些稳定反复出现的 weakness。这些 weakness 要么通过收集 human feedback 解决, 要么通过设计新的 environment 解决。

Charlie O' Neill: 一个可能比较有用的视角是: 我们到底愿意沿着技术 lineage 往回 rollback 多远, 然后从那里让模型 self-play? 极端情况下, 你可以想象: 只给模型 GPU, 再给它 neural network 这种最基础的东西, 然后说: “好, 现在自己研究出一个能训练模型解决这些任务的方法。”但现在实际发生的事情基本正好相反。我们总是待在技术 lineage 的最前沿。然后说: “这是 Anthropic 最近几个月刚发现的 training-stack bug, 把这些 bug 变成 environment。”你必须继续在 frontier 上训练和改进。所以实际上, 我们把之前整个技术 lineage 都固定下来了。当然, 也可以想象另一种世界。比如 rollback 到 GRPO 之前, 然后给它 environment, 让它自己重新发现最好的 RL 方法。甚至继续往更早的阶段 rollback。但我觉得 compute bottleneck 会非常强, 所以大家最后还是停留在 frontier。基本上就是: 把上一代模型之后, 人类新发现的 bug 和 improvement 做一个 diff, 然后把这些差异转成 training environment。

Dwarkesh Patel: 这其实还有一个好处: 每一代模型之间都能产生 non-stale 的新数据。某种意义上, 这就是 AI lab 内部的 continual learning。把过去三个月的 AI research progress, 通过 environment、RLHF 一类方法重新 distill 回模型本身。

Charlie O' Neill: 对, 本质上就是 distillation。也许这也是为什么有些人会觉得这条路线最后可能 asymptote。因为你始终只是努力把最近三个月的人类研究进展塞回模型。当然这些进展本身已经有 AI 参与。但 loop 里仍然有人类。感觉就像模型在一点点逼近人类研究者刚刚发现的新东西。

Beren Millidge: 不过有一点需要补充。如果你只是 distill trajectory, 那你当然很难超过 trajectory 本身。但 environment 不一样。你完全可以设计一个人类自己解决不了的 environment, 然后让 AI 去尝试解决。因此 environment 可以让模型远远超过“仅仅复制人类 AI researcher”。这可能才是真正超越人的路径。

Charlie O' Neill: 比如 RSI 的 training set, 你能给一个例子吗?

Dwarkesh Patel: 比如 Nanochat speedrun, 而且要求比最强人类 speedrunner 还快。

Beren Millidge: AI research 其实有很多 objective 是很好定义的。比如规定: loss 必须做到 1.3。现在没有人能做到, 但它是一个非常明确、可衡量、可验证的目标。AI 如果真的做到, 那就是做到了。

Dwarkesh Patel: 或者训练一个一亿参数的模型, 让它打赢 Minecraft。这个现在可能已经算太简单了, 但可以换一个复杂很多的游戏。

Charlie O' Neill: 想想也挺离谱的。现在我们居然会觉得“一亿参数模型打赢 Minecraft”太简单。五年前如果有人这么说, 完全不可想象。

John Schulman: 不过很多 research 其实不是这种形式。不是说你有一个 well-defined objective, 然后一直 hill climbing。更多时候是: 我们有一个 intuition, 觉得模型也许应该沿某个方向变好。然后又有一个 algorithm idea, 似乎能朝这个方向推进一点。接下来我们会设计一个 task, 专门看这个方法能不能出现一点 signs of life。如果看到了, 再逐渐把 task 做得越来越 realistic。

Dwarkesh Patel: 所以它更像 intuition-guided research。inner loop 的作用, 是验证这个 intuition, 而不是让 test 本身自动产生 insight。

John Schulman: 对。你并不是直接在优化最终真正关心的 objective, 也不是直接优化 production objective。你会先把 objective 放松一点。比如: 先牺牲一些 realism, 让某个方法在一个比较容易控制的问题上先工作起来。等方法成熟以后, 再慢慢把 realism 加回来。还有一种 research 更偏 explanation 和 theory building。Machine learning 里, 我们通常没有特别 predictive 的数学理论。但我们确实有很多非正式的 theory, 用来解释目前发生了什么。

Beren Millidge: 未来模型大概会在这些任务的混合上训练。一部分非常容易 verify。另一部分可能通过 LLM-as-a-judge 来评价。还有一部分直接问人: “这个结果看起来合理吗?” 希望最后这些训练能够泛化到那些更难、更模糊、更 fuzzy 的研究任务。我觉得一定程度上会。但它能不能泛化到足以让整个 loop 完全 self-sealing, 人类彻底退出, 我觉得目前还不确定。

00:33:51 - Long-horizon RL 会不会训练出 AGI?

Dwarkesh Patel: 我们稍微退一步。现在看起来, 接下来 AI research 的整体赌注大概是这样的: 我们会把 RLVR 扩展到数百万个不同 environment, 覆盖几百种 domain。最终模型会从里面学到一些基础 agent skill, 比如: 持续推进任务; 对信息和 context 做 triage; 最终学会和其他 agent end-to-end 协作; 等等。然后这种 agent 在 context 内的 sample efficiency 会非常高。你们也有人研究怎么把 in-context learning 扩展到越来越长。最后出来的东西, 基本可以作为一个 drop-in remote worker, 连续工作一周甚至一个月。第一, 你们觉得这确实是目前 lab 的主要赌注吗? 第二, 这真的足够吗? 也就是说: 模型只在 data center 里的 simulation 中学会“怎么学习”, 然后部署到真实世界以后, 却并不真正从 deployment 中持续学习。它只是在 simulation 里提前获得这些 meta-skill。

Charlie O' Neill: 现在已经很难区分, lab 到底有多少 effort 是直接投入 RSI, 又有多少只是为了做一个更强的 general model, 把它部署出去赚更多收入, 然后资助下一次更大的 training run。如果说的是后者, 那对, 我觉得大概就是现在大家的 bet。过去几年 environment 的演进方向已经非常明显。Anthropic 是个很清楚的例子。一开始全部聚焦 coding, 先把 coding 做到特别好。Coding 有很多现成互联网数据, 也有大量内部任务, 所以它应该算最容易构造 environment 的地方之一。然后再希望从 coding 里学到的长 task horizon 泛化出去。下一步是 finance。RL training 里塞大量 Excel。然后 PowerPoint。再继续覆盖整个 knowledge economy 的 long tail。这条路线看起来效果很好, 所以现在很多其他 lab, 包括 open-source lab, 也意识到这可能是正确赌注。

Dwarkesh Patel: 但这背后的含义是什么? 我之前采访 Dario 问过他: 如果你真的相信模型未来能像人一样 on-the-job learning, 那为什么还要把 PowerPoint 这种能力专门 bake 进模型? 按理说, 模型部署以后自己学会不就行了吗? 可能有几种解释。一种是: 我们相信模型很快就会拥有这种能力, 但现在还没有。那在此之前, 既然可以把这些 skill amortize 进训练里, 为什么不做? 另一种解释是: 其实我们并没有特别在意让模型立刻成为最好的通用 worker。我们真正关心的是 RSI。现在这些商业任务只是赚钱方式, 用收入去资助真正为 RSI 优化的下一代模型。等 singularity 真出现以后, 最后那个系统自然会解决今天模型存在的所有瓶颈。John, 你怎么看? 如果我们的目标是 generalization, 为什么现在的模型里仍然要塞这么多 task-specific knowledge?

John Schulman: 如果模型的 in-context learning 真足够强, 理论上你确实不需要专门在 finance 上训练。模型可以当场把相关书全部读一遍, 然后在对应 jurisdiction 里自己搞清楚应该怎么做。但即使模型已经聪明到可以 runtime 现学现用, 你仍然可能希望先通过 RL, 把很多 intuition 固化进 weights。因为这样 inference 更高效。现实里看起来, model provider 确实是在一个

domain 一个 domain 地加强模型。优先做价值最高、最常见的那些技能。我觉得这也是为什么最近模型明显变强的一个原因：高价值 domain 和最常见技能正在被越来越多地覆盖。

Beren Millidge: 还有一点，这两件事情其实可以同时做，而且成本没有高到必须二选一。模型很大，parameter capacity 很充足。从容量角度，它完全可以学很多东西。而且 domain-specific training 很可能存在 transfer。比如 finance 本身也许和 RSI 无关，但训练过程中学到的 meta-skill 可能相关：怎么判断什么重要；怎么形成 taste；怎么进行 long-horizon work。这些能力都可能 generalize。另外，世界上真正的 RSI 数据也没那么多。这些数据很难生成，而且成本很高。所以既然其他 domain 的大量数据也能带来部分 transfer，而且你已经有这么多 compute 和 parameter space，那当然可以一起训练。更不用说这些 domain 本身还有直接商业价值。

John Schulman: 还有一个更长期的问题：现在这种 sim-to-real paradigm 会不会永远占主导？目前的做法是，先观察 real-world task 是什么，然后尽量在 data center 里构造对应 simulation environment，再在里面做 RL。目前当然非常成功。但它也有很多 weakness。有些事情就是特别难模拟，尤其是涉及大量实时人类交互时。所以问题是：sim-to-real 会不会永远都是主流框架？

Beren Millidge: 在 sample efficiency 很低的时候，我觉得 sim-to-real 几乎一定必须是主流框架。因为现在训练一次能力，可能需要成千上万次 interaction。没有人类会坐在那里陪你完成几万次 RL interaction。所以只能先模拟，才能得到足够 sample。但如果以后 sample efficiency 大幅提高，我当然会预期 learning from deployment 的比例会越来越大。

John Schulman: 而且也不一定每次都需要重新 simulation。比如可以做 off-policy learning。把真实 deployment trace 拿回来以后，即使不重新跑整个 environment，也可能从这些 trace 里学到东西。

Dwarkesh Patel: 我想深入问一下这个问题。因为现在有一个挺奇怪的现象：可能 50% 的 compute 都用在 inference 上，但这些 inference 并不会直接让模型本身变得更好。理论上，digital mind 最大的优势之一应该就是：人类一生也许只有五十年真实经验。但一个模型通过所有 deployed instance，可以累计相当于几百万年的现实世界工作经验。而今天这些经验基本没有以非常直接的方式帮助同一个模型变强。感觉最终模型显然应该能够从这些数据里学习。一旦做到这一点，你几乎会得到一种部署在整个社会里的 intelligence explosion。因为同一个模型可以吸收无数 deployed instance 的经验。你们觉得这种 hive-mind 式的快速学习什么时候会出现？而且具体会怎么实现？

Beren Millidge: 如果拉到比较宏观的尺度看，我觉得这其实已经在发生了。只是现在的反馈发生在下一代模型。今天你完全可以把 deployment data 拿回来，做一些 filtering、judgment、annotation、synthesis，然后放进下一代模型的 pre-training 或 mid-training。

Dwarkesh Patel: 你觉得代际之间的模型进步，有多大比例来自这种机制？

Beren Millidge: 我觉得占比不小。当然我不知道各个 lab 实际上具体做没做，因为他们理论上都说不会拿用户数据直接训练。但中国公司肯定会大量利用类似数据。这种 advantage 很明显。某种意义上，这就是 distillation。让模型跑真实流量，从里面拿到一部分 deployment data，再用这些数据训练下一代模型。对于自己的模型，当然也可以这么做。

Charlie O' Neill: 我完全同意。如果把时间尺度拉得足够大，这件事情肯定已经在发生。我们通常想象的 continual learning 圣杯，是一种特别 organic 的 live loop：一个模型得到一条新经验，然后立刻在线更新，马上从中学到东西。一旦放大到这么细的 granularity，很多方法都会坏掉。但如果看 big lab 的整体流程，其实 continual learning 已经以更慢的形式存在了。Closed model 确实这么做。Open-source model 这边也已经有一些更高频率的早期例子。比如 Cursor 的 Composer。Harvey 在 legal agent 上也在做类似的事情。你先有一个模型。然后从真实 deployment 中拿到极其 task-specific 的 environment：用户抱怨什么；哪里经常失败；用户反馈里有哪些信号。这类公司反而有一个 big lab 不一定有的优势：它们可以非常有效地使用自己的 domain-specific deployment data。接下来它们会构造 environment，对 Kimi K3 之类的 base 做一次比较大的 post-training，然后再部署。中间也可能掺一些 online learning。Composer 当时就长期做过类似 REINFORCE 的 online update。当然现在仍然有人类在 loop 里。人类需要决定：“哪些 signal 是我们真正关心的？”“怎样从 deployment data 里构造 environment？”所以 cadence 还远没有你描述的那么短。但这个 loop 的确已经存在。之后只会越来越快。

Dwarkesh Patel: Composer 那个例子很有意思。Cursor 里用户会决定，要不要接受模型下一步 completion。以前 Tab model 就可以根据用户按不按 Tab 学习。每天根据这些行为继续变好——

Charlie O' Neill: 那个是老 Tab model。后来实际 generative model 也做了类似的事。

Dwarkesh Patel: 明白，有意思。

Charlie O' Neill: Online RL 有一个麻烦。你不像 group-based RL 那样同时有很多 rollout。你只看到：一个用户提出一个东西，然后模型得到一个 rollout。所以 variance reduction 是个很大的问题。Cursor 当时的解决方案比较 heuristic。他们会估计：这个 response 相对于平均水平到底好多少，或者差多少。然后根据这个估计做比较大的 REINFORCE update。至于怎么判断模型有没有被更新坏：如果新版在 CursorBench 上提高，就每五个小时 deploy 一次。如果没提高，就把这个版本扔掉。

John Schulman: 我觉得最大的困难其实是：你根本不知道 natural deployment data 的 reward function 应该是什么。如果只用一些 surface-level signal，比如“用户有没有接受这次 edit”，模型可能很容易 reward hack 这个信号。

00:45:24 - Sim-to-real gap

Dwarkesh Patel: 但这是不是意味着 sim-to-real 本身有一个更严重的问题？task horizon 越长，就越难在 data center 里模拟。甚至 coding 现在已经开始碰到这个问题。一个持续一年的 coding project，基本不可能完全不和 client、公司内部人员、真实用户发生交互。如果考虑最终希望 AI 能做的事情：运营一家公司；创建一家新公司并让它盈利；做 profitable day trading；赢一场官司。这些任务都非常难在 data center 里完整模拟。它们的学习过程本身就包含与现实世界持续交互。也许 sim-to-real transfer 足够强，模型能从模拟里学会这些事情。但可能不够。如果不够，那么模型就需要根据真实 interaction 做 weight update。一旦必须这样做，模型现在非常差的 sample efficiency 可能会变成一个真正根本性的 bottleneck。我之所以一直在问这个，是因为我的默认判断是：未来十年以内，很难想象完全不会发生某种疯狂的 recursive self-improvement。但如果真的有一个技术因素能阻止它，我觉得很可能就是 weight update 的 sample efficiency。人类从出生到成年看到的数据，和一个模型从 cold start 一直到训练完成看到的数据相比，模型可能差了六个数量级。所以问题有两个：第一，从 simulation 到真正极长 horizon、极复杂的现实任务，transfer 到底够不够强？第二，如果不够强，那么 sample inefficiency 会不会最终真的成为那个致命问题？

Charlie O' Neill: 我可能会这样区分：有些 task 模型会越来越强，有些 task 模型可能持续 struggle。关键在于任务到底是 cumulative，还是它面对的是一个持续变化的 non-stationary distribution，需要不断重新学习以前的很多东西。RSI 可能是 cumulative task。理论上，也许存在一个不到一百万 token 的 Python 文件，从零开始运行以后，就可以训练出能做 recursive self-improvement 的模型。每一个真正有效的 discovery，一旦被发现，就变成你永久保留下来的东西。比如如果 RSI 不需要重新发明新的 attention variant：那 attention 一旦发现了，就永久进 stack；Mixture of Experts 一旦发现了，就永久进 stack；GRPO 一旦发现了，也永久进 stack。以后训练新模型不需要重新发现 attention。你可能只需要跑 pre-training.sh、post-training.sh 之类的脚本。这就是 cumulative task。但现实世界大量工作不是这样。这也是为什么现在这么多人关心 continual learning。比如一个 law firm agent。它作为 legal associate 工作。这是一个非常 non-stationary 的 distribution。它必须知道公司里重要人物之间的关系，而且这些关系还不断变化。还要知道各种隐性的工作方式：事情通常怎么处理；去哪里找信息；谁负责什么。这些东西并不像 RSI 那样可以简单永久累积。所以我觉得以后会出现一种 task split。如果 lab 最终确认：RSI 确实是 cumulative 的，不需要重新发明整个 architecture，那他们可能会把越来越多 effort 和 compute 集中在 RSI，而不是其他更难 continual learning 的现实工作上。

Dwarkesh Patel: 这也太倒霉了。RSI 居然可能比当一个 paralegal 更容易。

John Schulman: 今天的模型在很多方面都弱于人。其中一些 weakness 可能确实和某种 regime 下的 sample efficiency 有关。有些 regime 里模型其实 sample-efficient 得惊人，比如 in-context learning。但也许存在一个 medium-length regime：在人类那里，我们可以通过某种高效的 weight update 很快学会东西，而模型做不到。这种 regime 下的低 sample efficiency，可能是模型 weakness 的来源之一。但还有一些 weakness 和 sample efficiency 完全不同。例如模型的 thought diversity 比人低。或者某些 long-horizon judgment 很差。大家经常说的 taste，我觉得其中有一部分其实就是：哪些行为从长期来看真的有效。人类通过经验已经发现了哪些做法长期有效。比如 software engineering 里，很大一部分 taste 其实就是：什么系统在这个 project 长期维护下去以后，仍然会保持 maintainable，而且不会不断出问题。所以限制 RSI 的 model weakness 有很多种。有的和 sample efficiency 有关，有的完全无关。

Charlie O' Neill: 可以做一个 thought experiment。假设模型有一万亿 token 的 context window。我们把你从出生到 RLHF 之前积累的全部 research experience 都塞进去。再假设它在 trillion-token context 下拥有和现在 million-token context 下同等的 sample efficiency 和 in-context learning 能力。那 taste 这个问题是不是就解决了？模型能不能做出和你当时一样的判断？还是说，即使把所有经验都放进去，仍然缺某种更根本的东西？

John Schulman: 模型还是需要学会怎样从 context 中学习。要么它得专门训练过：看到这些 context 后，应该产生什么 update。要么就只能依赖 generalization。

Charlie O' Neill: 所以不能只是把你整个人生、所有研究经验直接 dump 进去？

Beren Millidge: 你还是需要 long-context training data。就算理论上 context window 能做到一万亿 token，也需要对应尺度的数据去训练模型使用这种 context。现在模型只训练到 10k，你也不能突然塞一百万进去就指望它会用。

Charlie O' Neill: 对，我是假设这些条件都满足。

Beren Millidge: 如果都满足，我理论上觉得是可以的。最终问题其实是：taste 到底有多大程度可以从较短 horizon episode 中 meta-learn 出来？我觉得没有明显理由说明它必须来自特别长的 episode。人类本身也没有活一万年。但我们还是能比较快形成 taste。比如 PhD。一年级 PhD 和毕业后的 PhD 或 postdoc 差距很明显。但整个过程也就是五年左右。可能总共只做过十到三十个 research project。可是在这么短的一系列项目中，人已经能相当快地形成 taste。所以理论上模型也可能做到。AI 能拥有远远超过人类的经验数量，从中 meta-learn taste。真正没解决的是：这种东西能多好地 generalize 到特别 long-horizon 的任务。目前我们不知道。

Dwarkesh Patel: 回到刚才的问题。最终应该会进入一个 regime：AI 可以从每一个 deployed instance 里学到大量东西。现在可以说有一个很模糊的 meta-process：deployment 经验最终会影响下一代模型。但这个反馈 loop 非常弱。你们觉得那种真正快速的 hive-mind learning 会出现吗？如果会，具体怎样发生？

John Schulman: 我觉得我们最终能不能得到“从所有 deployment experience 中共同学习的 hive mind”，很大程度上可能是 incentive 问题，而不只是 technical question。公司未必愿意让 model provider 从自己的全部 deployment 中学习。因为这样可能等于把公司的 competitive advantage 交给模型提供商。

Charlie O' Neill: 我觉得经济 incentive 最先推动的，未必是对一个共享 base model 直接做 weight update。更可能是插入某种 module。最明显的例子就是 LoRA，但也可能是别的东西。已经有很多研究尝试：怎么把任意长 context 压进一个固定大小的表示。Linear attention 是其中一类。还有 cartridge，本质上可以理解成高度压缩、经过训练的 KV cache，用固定大小去承载大量信息。公司也许更愿意接受这种形式：这个 module 插进模型以后，可以学习我的数据，但不会直接修改底层共享 base model。实时从用户数据学习其实有很多不同实现方式。这种 module-based 方法，对 big lab 本身的 shared base model 帮助没那么直接。但经济压力可能会先推动大家走这条路线，而不是一开始就说：“把所有客户数据全部直接训练进同一个大模型。”

Beren Millidge: 但为什么经济压力会阻止这个？就算你现在有很多 cartridge、LoRA 之类的东西，也可以把最后收集到的 trace 全部放进下一代模型的 pre-training。

Charlie O' Neill: 可以。Big lab 还是能以更间接的方式从中学习，这当然也很有价值。但我很难想象第一步就是：“我们直接拿所有收到的数据不断训练同一个大共享模型。”

Beren Millidge: 对，我也觉得会分阶段。你刚才描述的方案，其实假设某天突然出现一个 discontinuity：continuous weight update 一下被彻底解决。现实里我觉得更可能是渐进式的。先用 cartridge 之类的东西在 deployment 里做 specialization。这些 specialized system 产生大量 trace。再把 trace 放回下一代 base model。三个月后，新 base model 在这些事情上更强。然后再 specialization。再 consolidation。循环越来越快。一开始每三个月 release 一次。然后每周。每天。最后每小时。当你做到每小时一次时，宏观上基本就相当于把这个问题解决了。

Charlie O' Neill: 这也正好回答你刚才问的：现在这套 paradigm 离真正 continual learning 还有多远？我们自己做过一点研究，其他人当然也做过很多。在非常 large scale 下，只要 batch 足够大、noise 被平均掉，这种 outer-loop 流程确实能工作：把新数据放进 mid-training，再自己构造 environment。某种意义上，这已经是 continual learning。真正的问题出现在 micro level。比如我现在只有一个模型，要为某家 law firm 持续更新。每次只有比较少的数据。我希望不断进行很小的 update。到了这个尺度，现有方法就开始大量失效。如果我只是拿 successful trace 去做 SFT，不论是 off-policy 还是 on-policy：当你连续做几百次这种 micro-update 以后，最终会看到 catastrophic forgetting。模型会忘掉很久以前，在 base model 上新学进去的那些信息。同时 general capability 也会开始下降。On-policy distillation 似乎可以把这个 horizon 往后推一点。但最终仍然会碰到类似问题。RL 很擅长把 capability 学进去。但它不太擅长把 explicit knowledge 写进去。比如：“这家 law firm 里这个人具体负责什么。”“这个公司有一个非常特殊的内部 process。”如果想通过 RL 把这种知识写进模型，你需要投入很多 compute 去设计对应 environment。

Dwarkesh Patel: 所以模型更新以后会在其他 skill 上变差，或者出现 forgetting。这个问题本质上到底来自 capacity，还是来自 technique？

Charlie O' Neill: 我觉得两者都有。SFT，甚至 on-policy distillation，都可能过于 destructive。RL 很好的一点是，它实际对模型的改动通常非常小。现在已经有不少 evidence 支持这一点。它只是让模型在一个很小的 loss valley 里稍微移动一下，到达一个更合适的位置。但这同时也限制了 RL。既然你只改变很少，那它真正能改变模型多少东西，也就有限。

Dwarkesh Patel: 所以你是说：为什么这里没有变成 winner-take-all，一个原因可能就是把那么多信息不断 distill 进同一个 base model，本身非常难？

Charlie O' Neill: 对。尤其是要迭代地做，而且不能破坏其他东西。

Beren Millidge: 但如果换一个新 base model，其实 distill 并没有那么难。所以这里我更倾向于 technique 问题。我不觉得是 capacity 不够。如果把所有新旧数据放在一起，拿一个完全相同大小的模型重新从头 pre-train，再把这些东西做进 mid-training，它大概率会更好。我觉得现在很多新模型其实就是这么来的。真正存在一个 bottleneck：为什么我们不能直接把同一个 base model 永远继续训练下去？为什么最后还是要旧模型积累的数据重新拿出来，训练一个新的 base？正如 Charlie 说的，这大概和 plasticity、catastrophic forgetting 都有关。如果你 naive 地不断在 non-stationary data 上训练：因为数据分布一直变，旧东西就会逐渐被忘掉。我们现在并没有特别好的方法阻止这种现象。

Dwarkesh Patel: 所以最终瓶颈可能变成：每次有大量新信息以后，重新从头训练模型的成本。

Beren Millidge: 对。而从头训练当然很贵。

Dwarkesh Patel: 但你本来不就会继续训练下一代模型吗？

Beren Millidge: 不一定。如果 continual learning 真解决了，也许最终根本不需要不断训练“新模型”。你可能永远只有一个模型，它持续学习、持续扩展。

Dwarkesh Patel: 但也许存在某种很深的技术原因，让这种事情本身就特别困难。

Charlie O' Neill: 对，这正是问题。现在我们确实已经把“必须从头训练”的位置往后推了很多。你可以从一个 pre-trained base 出发，继续做质量很高的 mid-training。甚至在后面的不同 checkpoint 上做一些 RL。这已经越来越像 continual learning。但它还远远不是：永远拿最新模型，做几个很小的 update，一轮一轮持续下去，而且永远不丢任何东西。

Dwarkesh Patel: 我有点困惑。因为这听起来不就是正常 training 本身吗？比如 post-training：你已经有一个经过大量训练的模型，然后再把某个 RL 过的 fork distill 回来。这不就是在对同一个模型继续更新吗？

Charlie O' Neill: 区别是，现在每次 update 的 scale 还是足够大。大量数据和大 batch 会把 noise 洗掉。而且训练不会只聚焦一个很窄的 distribution。正如 Beren 前面说的，如果每次只盯着一个 task——

Dwarkesh Patel: 但如果未来有几十亿 deployed instance，同时从所有 instance 学习，那理论上 noise 也会被平均掉。

Charlie O' Neill: 如果真到那个 scale，也许可以。

Beren Millidge: Charlie 说得对。Continual mid-training 确实可以持续很长时间。你也可以 rollback 到一个旧 checkpoint，再喂新的 mid-training data。但仍然不能无限做下去。如果一直对同一个 base 永久训练，最后某个阶段它还是会 asymptote。不能一直往那个 base 里塞新东西。这也是为什么今天大家最终还是会训练新的 base model。否则最简单的做法应该就是：永远对同一个 base 做 mid-training。但现实并不是这样。

01:00:33 - AI 的进步有多少其实来自数据？

Dwarkesh Patel: 我们聊一下 data。我一直很好奇：AI progress 到底有多少其实只是 data progress？这和自动化难不难是另一个问题。有没有可能存在某种 data distribution：只要拿今天的 architecture 去训练，就能直接得到一个在所有领域都全面超过 human expert 的 superintelligence？

Charlie O' Neill: 这里说的是 pre-training + post-training data，包括 environment 吗？如果都算，我觉得这种 distribution 理论上肯定存在。真正的问题只是：我们能不能构造出正确的 environment，把系统一路训练到那里。

Beren Millidge: 最 trivial 的情况是：直接把那个“能训练出真正 superintelligence 的 Python 文件”作为输出训练进去。模型只要把它 memorise 到 weights 里就行了。

Charlie O' Neill: 对。大概存在一条 RL environment ladder。如果你能一级一级构造出来，最终应该可以得到一个至少和 human researcher 一样强的 AI researcher。问题是：越往上，每一级 environment 的构造成本可能越来越高，甚至近似指数增长。所以未来进度有多快，本质上是：AI 爬每一级 ladder 的速度，和人类构造下一级 ladder 的难度，这两件事之间的 trade-off。现在 RL environment creation 其实还处于很早阶段。我们大量利用各种 asymmetry。比如一种常见 asymmetry 是：backward 比 forward 容易。什么意思？你可以定义一个非常复杂的数据生成过程。把其中某个 latent variable 隐藏，不告诉模型。作为 environment creator，你很容易从 process 往数据生成。但模型反过来必须花很多不可约的 token 和工作

量，才能推断这个 data-generating process 到底是什么。所以 environment 很容易造，但很难解。还有一种 asymmetry 是从现实世界注入信息。比如 Anthropic 可能通过几万个人和 LLM 的组合，在真实系统里发现一个 bug。然后把这个 bug 压缩成一个特别干净的 environment。单个模型理论上可以在几百万 token 里重新发现它。这些都是我们现在在主动利用的 asymmetry。然后希望 task-horizon generalization 能把这些训练推广出去。但我觉得迟早会出现 diminishing returns。首先 environment 本身会越来越难构造。因为你不可能永远找到这种“反向容易、正向难”的巧妙 process。最后必须真的构造一个像长期人类工作那样的复杂 task。那会非常难。其次 agent 真正运行这些长期 task 本身也会有 compute 和 time bottleneck。所以最终这条曲线应该会开始变平。

John Schulman: 我最近看到一个挺夸张的例子。有人拿了一个只训练到 1930 年数据的 Talkie model。也就是说，它实际上完全不知道现代 programming。然后只用现代 coding-agent data 对它做 fine-tuning。最后它在 SWE-bench 上居然超过了 Claude 3 Opus。一个几乎没有现代 coding 知识的模型，只用适量的数据，就能表现得比一个大很多的 pretrained model 更像 coding agent。这说明一个很有意思的事情：只要你已经有了了一份正确的 expert behavior example，把这种行为复制进一个弱很多的模型，有时比我们想象中容易得多。

Charlie O' Neill: 但最近也有一个很好的反例。有篇论文先把模型只训练到小学五年级数学，以及小学水平英语。所以它本身是一个还算正常的 language model。然后研究者试图直接用 RL，把它训练到高中后期甚至大学数学。结果 gap 太大，模型根本爬不上去。但如果中间逐级加 curriculum：七年级、八年级、九年级……一直往上，最后当然就可能爬到十二年级。所以还是那个问题：这条 ladder 上每一级之间的距离到底有多远？而且构造这些 rung 到底有多难？

Beren Millidge: 这其实还是 RL signal 的问题。今天的 RL 并不擅长 exploration。如果一个模型在 128 个 rollout 里都完全碰不到正确解，它基本就得不到足够 signal 去进步。所以 RL 特别依赖 curriculum。而 pre-training 没这个问题。

Charlie O' Neill: 对。Pre-training data 和 post-training data 本身就不一样。随着时间推移，人类参与可能越来越少。但这并不会改变一个事实：你最终还是被“现实世界里到底有多少新 signal 可以提取”卡住。现实世界当然有很多 signal。有人每天在做 spreadsheet task。有人在做 legal task。各种东西都有。但现在模型能力已经越来越接近 frontier。这时候真正有多少现实中的 bits，能推动 frontier model 再提高？比如：现在有多少新数学问题，难度刚好超过当前模型，但又已经被解决了？有多少新的 coding problem，超出了当前模型能力，同时又在现实中有人解决？我觉得 diminishing returns 就来自这里。即使把整个现实世界都拿来作为数据源，也未必存在足够多恰好能把模型推入下一个 capability basin 的 bits。

Beren Millidge: 完全同意。最终就是 signal 到底从哪来的问题。在 pre-training 阶段，signal 本身已经存在于 Common Crawl。你不是没有 signal。真正的问题是：signal 淹没在大量 noise 中。所以主要任务是 filtering。而 filtering 是非常可以自动化的。但模型越来越强，进入 mid-training 和 post-training 以后，情况会发生根本变化。你真正需要的 signal，可能根本不存在于原始数据里。无论怎么过滤 Common Crawl，都不可能突然过滤出一个还没人发现的 Millennium Prize Problem 证明。这时候你必须从其他地方获得新的 bits。可能来自人类专家，直接让他们写 reasoning。可能来自人类设计 environment 和 objective。也可能来自真实 deployment 产生的数据。总之这些 bits 必须从某个地方产生出来。

Dwarkesh Patel: Pre-training 里面也有一个问题：过去这些 progress 到底多少来自 data？我之前和 Princeton 的 Jerry Han 做过一个实验。我们把 2019 年到现在不同年代的 training recipe，和不同年代的数据集做两两组合。比如拿 GPT-2 recipe 去训练今天的 Ultra-FineWeb。再拿最新 open-source training recipe Delphi 去训练老的 The Pile。做完整个 grid。然后看：为了达到同一个 capability level，不同组合各自需要多少 compute。在小规模实验里，结果大概是：data improvement 带来了约 12 倍 compute efficiency gain。architecture improvement 大概是 3.7 倍。如果这个现象在大 scale 仍然成立，也就是大部分 pre-training compute-efficiency progress 其实来自更好的 data：这种趋势还能持续多久？还能不断做更强 filtering，继续增加 synthetic data 吗？你们觉得 pre-training data 带来的提升还有多少空间？

Charlie O' Neill: 我的 prior 是：低垂果实大概已经吃掉很多了。互联网本身给了我们一个巨大的 data block。但互联网并没有以同样速度持续增长。真正有用的内容增长得更慢。也许以后还能有不少 0.1% loss improvement。但我觉得不会再出现和过去一样多的大收益。不过你刚才说 architecture 和 data 合起来大概有 33 倍 cumulative improvement，这一点也很有意思。我记得 Epoch 之类的估计是：从 2019 年以来，algorithmic efficiency 大概每年提高 3 倍。如果真是这样，累计起来应该有几千倍。所以这里少掉的那一大截到底从哪里来？一个很自然的解释就是：大量 improvement 实际来自 post-training。

Dwarkesh Patel: 我觉得另一个解释可能是：很多 compute-efficiency improvement 本身具有 scale dependence。而我们实验做得非常小。于是问题变成：data improvement 和 algorithmic improvement，哪个随 scale 增长得更多？我们没有足够 compute 去测。

Charlie O' Neill: 如果只从理论直观看: architecture 的 scale dependence 相对比较清楚。通常可以拟合出不错的趋势。但如果把 pre-training、mid-training、post-training data 全部混在一起, 我完全不知道应该怎么拟合。

Beren Millidge: 有意思的是, 我反而觉得 data 可能随着 scale 越来越重要。Architecture 很多时候像一次性 unlock。简单地说“architecture 带来了 X% efficiency improvement”有时会误导。因为某个 architecture 真正做的事情, 可能是让你进入一个以前根本到不了的 qualitatively different regime。进入那个 regime 以后, 具体能力最终又主要由 data 决定。比如如果没有 GQA, 永远用 full attention, 那 million-token context 会贵得离谱。那你就根本用不了 million-context 的数据, 也不可能学到对应能力。如果你只在 2K context 下测, 这种 architecture 根本没有 unlock 新东西。这时候看起来 data 的贡献当然比 architecture 大很多。所以我不确定把这些收益直接当 multiplicative factor 相乘, 到底有多合理。

Dwarkesh Patel: 那你怎么看 data 的 scale dependence?

Beren Millidge: 我觉得现在很多 mid-training 和 post-training data, 反而在更大模型上价值更高。尤其是 long-context、long-horizon environment data。这些东西需要足够大的模型才能真正利用。你如果拿一个一亿参数模型去训练 SWE-bench trajectory, 它基本学不出来。同样一份数据, 放在一个真正有合理规模的模型上, 收益可能完全不同。

Charlie O' Neill: 现在 architecture change 也越来越难独立评估了。比如 Kimi、DeepSeek 做 architecture modification, 目标已经不只是降低 pre-training loss。他们还会直接考虑 deployment。比如 inference efficiency。DeepSeek 的某些 compressed attention 设计, 本身不一定是在追求一个更加 fundamental 的 Pareto improvement。更多是在问: 模型以后真实使用时, 怎样才能更便宜、更高效?

Dwarkesh Patel: 还有一个我很好奇的问题。现在越来越进入 RL-heavy regime 后, parameter scaling 会怎么发展? 看 open-source architecture, 过去 frontier model 的 parameter count 也许大概每年翻倍。即使 closed frontier model 可能只有 100B 或 200B active parameter: 这种数字还会每年继续翻倍吗? 现在大量 compute 要花在 RL rollout 上。Inference efficiency 越来越重要。也许还存在某个 threshold: 当 capacity 足够以后, 继续无脑扩大参数并不会带来那么多价值。你们觉得到了 2030 年, 一个 frontier model 会有多少 active parameter?

Charlie O' Neill: 我觉得未来几年不一定继续大涨。因为现在大家非常集中在越来越长 horizon 的 RL rollout。Inference efficiency 非常重要。而且模型本身处理这些任务的能力似乎还没有 saturate。真正的 bottleneck 反而是 environment。所以 parameter count 可能会稍微 plateau 一段时间。我有一种感觉, Mythos 和 GPT 系列可能比外面传的“十万亿参数”小得多。单纯和 open-source model 做一些粗略比较, 也许就能看出来。未来几年我不太预期 parameter 会暴涨。但这里有很多 trade-off。模型大小最终要看: pre-training data 有多少; 最难 RL environment 有多难。最理想的 model size 是: 在你最难的 environment 上, 至少能有一个合理的 pass@1。如果模型已经足够大, 可以通过那些 environment, 再继续做大只是在 rollout 时烧更多 inference compute, 就未必划算。所以很大程度上取决于 Mercor 和 lab 内部团队能多快把 RL environment complexity 推高。

John Schulman: 我还是预期模型会继续变大。因为整体 compute 还在增加, GPU 也越来越强。但具体增长多少, 要看一些不太直观的 scaling law。一个变化是: 随着 high-quality pre-training data 越来越稀缺, data efficiency 可能会比单纯 compute efficiency 更重要。这也可能影响模型应该做多 sparse。另外, 我们其实并没有真正理解 sparsity。Total parameter 和 active parameter 是两个不同资源。近年来 sparsity 确实越来越高。但我不确定它会无限增加。可能存在一个 sweet spot。甚至有一种 argument 是: sparsity 可能让 data efficiency 变差。因为相同知识可能要在多个 expert 里重复学。当然这个观点有争议。总体来说, 我们对 scaling law 的理论还不够好, 不知道为什么 sparsity 有效, 有效到什么程度, 也不知道它最终会不会在某个 sparsity level plateau。

Dwarkesh Patel: 你能具体解释一下 data efficiency 对 parameter choice 的影响吗? 听起来你的意思可能是 sparsity 应该下降。除此之外还有什么?

John Schulman: 核心是: 以前做 scaling law optimization 时, 主要优化的是 compute efficiency。你有很多 architecture choice。每种 choice 对应一条不同 scaling law。最后画 performance vs compute, 选 envelope 上最好的模型。但如果以后 data 才是真正的稀缺资源: 那 x-axis 就不是 compute, 而是 data。我们可以假设 compute 很充足。这时 optimal architecture 会改变。最后落在 frontier 上的模型集合, 也会不一样。

Charlie O' Neill: 另外, 我也不觉得过去几年 model size 真的一直每年翻倍。人们训练 trillion-parameter model 已经很多年了。以前甚至有 Falcon 这类 open-source 大模型。Liam Fedus 之前也提过, 他们早期做过一个一万亿参数、非常 sparse 的模型。

John Schulman: 那其实是他们在 OpenAI 之前，在 Google 做的 Switch Transformer。

Charlie O' Neill: 对。那种模型 knowledge 很强，但 reasoning 很差，因为太 sparse。所以感觉过去几年大家其实一直在 100B 到 2T parameter 这个范围里探索。并没有一条漂亮的线性增长。

Beren Millidge: 我觉得这里有两个因素。第一，Charlie 说得对。RL rollout 对 inference efficiency 的要求非常高。这会强烈压低 active parameter。第二，total parameter 能不能继续扩大，很依赖 hardware。真正 serve 一个 multi-trillion-parameter model，需要非常高的 memory bandwidth 和 VRAM。现在大量 infrastructure 还在用 H100。等大家逐步迁移到 GB 系列，再到 Vera Rubin，模型在 RL inference 时可以承载的 parameter scale 才会继续提高。Data 这边也很有意思。直觉上，更大的模型在 data point 上往往更 sample-efficient。哪怕你没有把模型完全 train saturated，同样数量的数据，大模型也可能 generalize 更好、达到更低 loss。现在的问题是我们还有很多 data，真正稀缺的是 compute。所以大家更倾向于用 inference-efficient 的小模型。但如果以后 compute 不再是主要 bottleneck，而 data 变成 bottleneck：那可能又会重新倾向更大的模型。即使这些模型 under-trained，也可以靠更大的 parameter count 获得更强 generalization。

Dwarkesh Patel: 但如果只看最基础的 Chinchilla scaling law，把 parameter 推到无限大，对达到同样 loss 所需 data 的减少其实很有限。我记得最多也就不到一个数量级。这是 power law 本身决定的。

Beren Millidge: 但现在我们其实处于相对 Chinchilla 过度训练模型的一侧。也就是说，我们目前通常会给模型喂比 Chinchilla optimum 更多的数据。如果未来 data 真开始稀缺，我们完全可能重新回到 Chinchilla-optimal point。甚至走到 model-undertrained 的一侧。

Charlie O' Neill: 不过即使未来新芯片都上线，接下来几年 compute 应该还是非常紧缺。所以未必这么快就会切换到那个 regime。

Beren Millidge: 最终看的是 training compute 和 inference compute 的比例。如果 data 是 bottleneck，而 compute 不是，那应该把模型做得更大。如果 compute 是 bottleneck，就应该倾向更小的模型。而且你还可以用模型生成 synthetic data。所以最终很难预测。

John Schulman: 我觉得当初 scaling law 之所以花了那么久才被真正搞清楚，也有这个原因。如果其他东西没配对，曲线根本不会那么漂亮。我们后来看到图里那些极其干净的 straight line，其实隐藏了大量细节。Hyperparameter 必须按正确方式 scale。Optimizer parameterization 也要设计得好。最好做到 model size 变化时不需要重新手调一堆 hyperparameter。否则 scaling law 看起来不会那么干净。

Charlie O' Neill: Bug 其实也有自己的“scaling law”。比如 Kaplan 当时 cosine annealing 的问题。还有 embedding parameter 当时好像也没有完全正确算进去。在小模型里 embedding parameter 占比不低，所以会让 estimate 偏掉很多。

01:18:03 - 为什么 RL 实际上这么有效？

Dwarkesh Patel: 聊一下 RL。一年前，很多人还在说 RL 可能很难 scale 到特别强。John，你之前写过一篇文章提到：在一次 RL episode 里，模型可能只学到一个 bit。也就是：“答案对了还是错了？”我今年也写过一些东西说：实际情况甚至更糟。如果 pass rate 很低，模型几乎不可能采样到正确答案，那一个 RL episode 基本学不到什么。但看看今天的模型，它们明显变聪明很多。而且似乎很大程度上来自 RL scaling。Beren，你前几周也写过一篇文章解释这个问题。为什么 RL 最后比最初的信息论直觉成功这么多？

Beren Millidge: 我觉得有几个原因。首先，一个经常被低估的东西是 mid-training。很多大家归功于 RL 的成功，其实很大一部分来自非常好的 mid-training data。基本上就是：先用 synthetic reasoning data 和各种 environment data 做类似继续 pre-training 的过程。这会把模型 warm-start 到一个特别适合 RL 的位置。很多时候，mid-training 已经把模型推到最终 RL checkpoint 能力的大约 80%。然后 RL 在这个基础上主要做的是 policy tweaking。这也是为什么它实际不需要那么多 bits。模型不是从零学习所有 behavior。它只需要通过每个 episode 提供的少量高价值 bits，对已经存在的行为做选择和强化。而且这些 bits 的信号极其干净。相对普通 pre-training，它的 signal-to-noise ratio 高很多。这也是为什么不能简单说：“那我把 successful reasoning trace 拿出来做 SFT 不就行了吗？”

Dwarkesh Patel: 因为 RL 得到的正好就是“怎样才能得到正确答案”的那部分 bit？

Beren Millidge: 有两个层面。第一，确实如此。它告诉你的 signal 直接和最终答对有关。但 SFT 的 trace 里其实也包含这个 bit。比如你有一整段数学 reasoning，最后有 answer。SFT 同样会训练 answer token。真正关键的是：RL objective 会忽略掉其他大量 bit。SFT 要求你尽量匹配 teacher 生成的具体 reasoning token。等于你必须学：这个 teacher 恰好怎样一步一步推

理。这给了模型很多额外信息，但其中大量信息未必是真正和任务成功相关的。RL 反而只保留最后那个极少量的高价值 signal。因此真正重要的 signal 不会被其他大量 token-level detail 淹没。从 training step 的角度，这会极大提高 signal-to-noise ratio。所以 RL 可以非常高效。

Charlie O' Neill: 过去一年大家一直在争论：RL 到底在模型里做了什么？和 mid-training、SFT 相比有什么不同？有一个很常见的讨论是：RL 后 pass@1 上升，但 pass@256 可能下降。因为一些非常罕见的 correct reasoning trace 可能被 down-weight，最后被大量更简单的 reasoning trace 的 gradient 压过去。我觉得现在有一个比较简单的理解：只要有足够 compute，能用足够大的 group size sample，那么即使某种正确行为概率不高，只要在 group 中出现的概率达到一个非忽略水平，它最终还是会被 up-weight。而且正如 Beren 说的，mid-training 和更多 pre-training 已经提高了 RL 的 starting point。也就是提高了 pass@1。这个 starting point 会随着 pre-training token 增长。

Dwarkesh Patel: 我想问一个很基础的问题。这个解释逻辑上我能理解，而且也许有 empirical evidence。但看看模型本身，我还是会觉得过去一年发生了某种非常大的 qualitative change。也许 RL 只是把模型本来偶尔会做出的正确 reasoning policy 给 up-weight。理论上两者并不矛盾。但如果 RL 对模型的 parameter 改动其实这么小：我们应该怎么解释模型在 qualitative capability 上看起来提升得这么大？

Beren Millidge: 这里有一点很重要。“RL 只得到很少 bits”“parameter 只改了一点点”，并不意味着 RL 的 behavioral effect 很小。你可以只用少量信息，只做小 parameter update，却让 function space 发生非常大的改变。一个 bit 从信息论上就可以排除一半 hypothesis space。这其实已经是很大的变化。所以如果模型本身起点已经非常好：small number of bits, small amount of RL，并不意味着最终 behavior 不会发生 dramatic change。

Charlie O' Neill: 我觉得可以从两个角度看。第一，大家最开始很期待 RL 能带来很强的跨领域 reasoning generalization。比如只在 math 上做 RL，然后 coding 也一起变得特别强。我觉得我们并没有看到那么强的 horizontal generalization。如果希望 code 真的很强，还是得做 coding environment 上的 RL。但我觉得我们确实得到了另一种东西：horizon generalization。模型学会了使用更多 token、持续更长时间，而且仍然可以继续推进任务。你可以在越来越长的 environment 上训练，然后把模型放到完全新的 environment。它也许没有泛化出解决那个新任务所需的具体 reasoning pattern。但它至少已经学会：怎样在一个 task 上持续工作更长时间，而不是很快失去 progress。这件事情本身就成功率高度相关。EdgeBench 有篇论文显示，模型能够有效工作时长的增长速度大概每三个月翻倍。这至少说明某种 horizon generalization 确实存在。第二，可以类比 pre-training 里的 quanta。Pre-training loss curve 看起来非常 smooth。但如果看模型内部真正发生的事情，它其实是在一个一个学习离散 capability。比如：之前没有 induction head，突然某个阶段就出现 induction head。类似的离散能力可能有成千上万、上百万甚至更多。把所有这些 phase transition 平均起来以后，宏观 loss curve 就变得非常平滑。RL 也可能有类似现象。我们有一个很慢的 outer loop。训练一个 model。对它做 RL。然后下一代模型训练时，把上一代产生的 synthetic reasoning trace 放回 mid-training data。于是我们不断在各种不同 task 上跨过这些 quanta。单独看一个 task，也许是非常突变的：某个 finance task 的 pass rate 从 0.5% 直接跳到 90%。Excel task 也是。其他 task 也一样。最后把所有任务平均，再加上 horizon generalization：你从外部看就会觉得：“这个模型怎么突然 qualitatively 强了这么多？”

Beren Millidge: 而且 RL 也不是完全没有 cross-domain generalization。Math 和 code 之间还是会有一些 transfer。Puzzle 和 math 也会互相 transfer。但我觉得更重要的是：今天大家训练的 environment 数量和覆盖范围，本身就比较两年前大太多。以前你在日常生活里遇到一个奇怪 task，lab 根本不会专门为它训练模型。现在很多这种具体 task 都已经有了对应 environment。所以模型自然在日常使用里表现得更全面。

01:24:54 - Move 37 与 entropy collapse

Dwarkesh Patel: 我们前面聊过 RL 可能导致 entropy collapse。也就是说，模型逐渐把 probability mass 集中到 base model 本来就会的少数 solution 上。Policy update 也比较 sparse。但 RL 还有另一条历史叙事。从 Atari，到 AlphaGo 的 Move 37。Move 37 是一个非常具有创造性的动作。AlphaGo 并没有被初始化成人类棋手数据的简单模仿者，却可以找到人类之前根本不会走的策略。所以什么时候我们应该期待 LLM 的 RL 也产生类似 Move 37 的东西？也就是极端的、甚至超越人类的 creativity。因为这种 intelligence 在某种意义上可以 de novo 地探索。

Beren Millidge: 这里有几个点。首先 AlphaGo 用了 MCTS。它的 exploration 显然比普通 policy gradient 强很多。所以不能简单把 Move 37 全部归功于普通 RL。但我也觉得 RL 必然会消灭 creativity。当然下面这个例子很 qualitative。但比如 OpenAI-Hugging Face 那次事情里，模型为了突破 sandbox，一次能找到多个 zero-day。这显然已经有某种 Move-37-style 的 creativity。而且它主要来自 LLM 本身的 generalization。所以至少在 long horizon 上，并不能说 RL 把 entropy 全部压没了。

John Schulman: 大家说 creativity 时，其实经常混合了两种不同的东西。一种 creativity，本质上就是解决非常难的 search problem。Move 37 显然是这种。或者让模型写一首必须同时满足大量约束的诗。如果专门训练，我觉得 AI 以后在这种 creativity 上会非常强。但另一种是 output distribution 本身的 diversity。这方面 RL 后的模型确实通常明显下降。模型会形成固定的 tics。看起来它们写作能力不错。但如果做 distributional analysis，会发现模型不断重复相同 theme。经常用相同 character name。经常回到相似写法。和 human author 相比，真正的 diversity 低很多。你得到的是一种很强、很 polished 的 style。但不是很多真正不同的 style。所以这种 diversity，我觉得确实被 RL 压掉了不少。而且我们刚才还聊 distillation。现在很多 open-weight model 大量从 Claude distill。结果所有模型写出来都越来越像 Claude，也继承同一批 tics。这种 monoculture 我觉得挺令人担忧。

Beren Millidge: 但我还是觉得这是 RL 这个方法本身必然导致的。Distillation 也一样。Distillation 本质上只是在学数据。如果你的 data 本身不够 broad，这并不说明 distillation algorithm 有根本问题。很多 RL entropy collapse，我觉得其实来自：verifier 太简单，environment diversity 不够。比如 writing 可能由某个 judge 打分。Judge 自己有一些稳定的偏好。模型学会 reward hack 这些偏好。于是输出越来越集中。这主要是 judge 的问题。不一定是 RL 本身的问题。

01:28:32 - 快问快答：时间线

Dwarkesh Patel: 最后快速问几个 timeline。第一个：什么时候会有这样一种模型，用户感觉就像真正雇了一个可以直接使用的 remote worker？而且不是只有 coding。还包括 video editing、law、paralegal 等各种 white-collar work。它能完整使用 computer。连续一个月无缝学习、工作。完成复杂 project。中间还需要和其他人发生交互。基本上一个人类 worker 在一个月里能做的事情，它都能做。

Charlie O' Neill: 如果限定必须像今天的人一样，通过现有 computer interface 做全部事情，我觉得也许三到五年。但如果允许公司把 organization 和 software 都重新设计，让 AI 更容易操作，也许一年左右就能做到相当大一部分。很多系统本来就会开始给 AI 提供 API。大家会故意重新设计 workflow，让 agent 更容易使用。

Dwarkesh Patel: 也就是说，一年左右可能已经能达到一个不错的版本？

Charlie O' Neill: 对。我觉得那时可能会有一种“基本可用”的版本。但完整 generality 还会更久。

Beren Millidge: 如果说真正 full generality，我可能会说三年左右。但 Charlie 说得对。大量组织会主动让自己的系统变得更适合 AI 使用。所以在达到完全 generality 以前，可能已经能做到 80%-90%。

Dwarkesh Patel: 但一年和三年之间具体差的是什么呢？

Beren Millidge: 我觉得会存在很长的 miscellaneous long tail。有各种很奇怪的小事，人类可以处理，但模型要花相当久才能全部覆盖。

Dwarkesh Patel: 你说的主要是 computer-use 的问题，还是基础 cognitive capability？

Beren Millidge: 我觉得核心还是：我们能多快解决这种 online learning。以及是否能仅仅依靠 compaction、把信息写进文件之类的方法，就先解决 80%-90%。这是我最大的不确定性。我真的不知道。

Charlie O' Neill: 举个它可能还不擅长的例子。有时候工作里你必须真的去催一个人。甚至得表现得比较强硬，逼对方把事情做出来。模型大概不会这么做。它可能会太客气。

John Schulman: 我觉得还要考虑一个问题：人类 remote worker 的质量本来就差异极大。如果你去 Upwork 找人做 software engineering project，质量分布非常宽。很多时候你很难让对方把事情做好，或者认真吸收你给的所有 feedback。我甚至觉得某些场景里，pre-AI 时代能找到的人类 remote worker，已经比今天的 AI 差了。所以这个问题会有点复杂。某种低质量工作意义上，我们也许已经有“remote worker”了。但在很多高质量工作上，显然还没到 human level。总体上我同意 Charlie 和 Beren。可能一年左右，就会有某种形式已经挺像样。它会有一个明确的 remote-worker form factor。有些事做得特别好，有些事还不行。然后继续提高。

Charlie O' Neill: 我们一直会根据 long tail 移动 goalpost。你以前好像提过报税这个例子。今年我自己就是直接告诉 Codex：把报税需要的所有东西都给我找出来，然后发给会计。它要在电脑上点很多东西，下载一大堆文件。任务列表很长。结果它全部做完了，而且非常好。很多这种工作今天其实已经能做。

Dwarkesh Patel: 好。下一个：什么时候 AI 能给你带来 10 倍总体 productivity uplift？也就是说，本来需要一年才能完成一个 breakthrough，现在一个月就能完成。

John Schulman: 我不太愿意给这种东西一个单一 scalar。因为某些类型的工作可能已经超过 10 倍了。比如某些数学工作——

Dwarkesh Patel: 抱歉，我说得更具体一点。就以你们 AI researcher 为例。目标是推进 state-of-the-art AI research。什么时候 AI researcher 的总体工作效率能提高 10 倍？

Charlie O' Neill: 大概五到十年？

Dwarkesh Patel: 这么久？那还挺远。

Beren Millidge: 真的？你觉得这个反而比 general remote worker 更晚？挺有意思。

Dwarkesh Patel: 我现在意识到，我们对“fully general remote worker”的定义可能完全不一样。我刚才应该说清楚。

Beren Millidge: 对。因为 AI researcher 本身当然也是 remote worker。

Charlie O' Neill: 我刚才想象的是：普通 white-collar work，在一个月 horizon 内完成。超过两个月以后，我觉得它和真正研究型工作就开始拉开差距。

Dwarkesh Patel: 也就是一个很 competent 的 white-collar worker。但不要求是非常有创造力的 researcher。

John Schulman: 那如果说 AI researcher 10 倍，我会说两年。

Dwarkesh Patel: 两年？10 倍？好。Beren 呢？

Beren Millidge: 这么想的话，我其实觉得也有可能。因为现在在 coding 这种局部工作上，AI 带来的 uplift 肯定已经超过 10 倍。如果它能自主跑哪怕一两轮 experiment-feedback loop：跑实验，看到结果，自己决定下一步，再跑第二次。光这个就已经会带来非常大的 uplift。

Dwarkesh Patel: 所以两年以后，AI researcher 生产率提高 10 倍。如果把这个数字放进一个非常 naive 的 AI progress model：假设相当一部分 AI progress 本来就来自 AI researcher。现在他们突然生产率提高十倍。那两年以后 AI progress 的速度可能会出现非常大的加速。

Beren Millidge: 我觉得这更可能意味着：AI progress 不再被 researcher 跑小实验的速度 bottleneck。然后 bottleneck 会转移到其他地方。

Dwarkesh Patel: 当然。但即使 bottleneck 转移，很多事情还是会快十倍。而且那也会让下一个能带来 100 倍 speedup 的东西更早出现。

Charlie O' Neill: 这个我还是愿意给更长一点的时间。

Dwarkesh Patel: 你的核心顾虑是什么？

Charlie O' Neill: 我自己吸收所有新信息，并根据这些信息为下一个 experiment 做出接近 Bayesian-optimal 决策的能力。

Beren Millidge: 我的假设是：这里面有一部分也可以 delegate 给 AI。AI 现在已经逐渐能做这种 decision。它跑了一个 experiment。拿到 result。然后自己决定下一个 experiment。只要它能连续做两三轮而不出问题，这已经会带来很大的 uplift。

Dwarkesh Patel: 最后一个问题。什么时候会出现这样一种 AI：在所有可以通过 computer 完成的工作领域，都全面超过最顶尖 human expert？不只是 AI research。而是所有 cognitive work。而且不是 short-horizon。即使一个任务本来需要三年，AI 最终也会比人做得更好。

Beren Millidge: 这基本就是 ASI 了吧？

Dwarkesh Patel: 对。

John Schulman: 我会说三到四年。

Dwarkesh Patel: 什么？这么快？虽然我也不能说这一定不对，但是——

John Schulman: AI research 现在得到的 attention 显然非常多。它确实属于比较难的事情之一。但同时它又不是对 AI 最不友好的领域。因为里面包含大量 code 和 math。这些刚好是模型非常擅长的东西。如果是涉及 3D、spatial reasoning、physical world 的任务，我觉得会慢一点。比如 mechanical engineering。而且这种领域现在得到的关注也没 AI research 那么多。所以可能更晚。

Dwarkesh Patel: 但我的定义也包含那些数据很少的领域。AI 必须能现场把一个新领域学会。比如它必须最终比 TSMC 最好的 engineer 还强。

John Schulman: 那你必须假设：AI 至少能拿到和人类 engineer 一样的 onboarding material。然后 long-horizon learning 还得进一步被解决。

Charlie O’ Neill: 我会说五到十年。

Dwarkesh Patel: 所以你的意思是：自动化 AI research 基本已经接近 ASI-complete?

Charlie O’ Neill: 对，我差不多是这么想。现实世界里有很多东西，即使模型外面已经有 memory system，即使 context length 再变长一点，还是很难。很多任务即使允许模型自己 research information、自己记笔记：按今天的形式，你仍然可能需要超过一百万 token 的有效 context。

Beren Millidge: 如果只说 lab 现在重点投入的那些领域，我大概也同意五年左右。但现实世界有一个非常长的 long tail。理论上 AI 也可以出去学那些东西。但很多领域现在根本没人专门为 AI 做数据、做 environment，也没有投入 compute。所以如果要求“真的超过每一个领域里的所有 human expert”，那 long tail 可能会拖得更久。

Dwarkesh Patel: 我刚才的定义还包括：AI 学一个新 domain 的速度，也至少和人一样快。

Beren Millidge: 我觉得这甚至未必是必要条件。因为 AI 本身最终拥有的累计经验，可能远远超过任何一个人类。

Dwarkesh Patel: 非常感谢你们来聊。我觉得这种形式很好。不同研究者可以直接互相不同意、争论，然后一起把问题讨论清楚。今天非常有收获。